PromiseTracker System Specification

Version 3.0 · Status: draft · 9 November 2026

Supersedes *PromiseTracker System Specification v2.0* (reference architecture, never implemented).

Contents

1. Executive summary
2. System architecture
3. Plugin architecture
4. Data models
5. Pipeline architecture
6. Error handling standards
7. Frontend architecture
8. Testing requirements
9. Security requirements
10. Deployment and operations
11. Performance requirements
12. Development workflow
13. API documentation
14. Design decisions
15. Appendix A: items to confirm

Conventions used in this document:

- **CONFIRM** marks a statement that must be verified against a running system before implementation. Appendix A collects all of them.
- File paths such as `src/utils/ssrf.ts` refer to the v2 repository, cited as evidence or as code to be reused. They are not v3 paths.
- "Core" means `plugin-core`, the plugin that ships the domain model.

---

1. Executive summary

1.1 Purpose

PromiseTracker tracks what politicians and public institutions promise, records whether those promises are kept, and publishes the record for citizens. The system ingests source material in many formats, extracts candidate claims with AI assistance, routes them through human verification, and serves them on a public site with an embeddable widget set and a read-only API.

v3 is a re-architecture of the v2 system. It has two defining properties.

Everything is a plugin, including the core. A thin kernel defines five port interfaces. A core plugin supplies the entire domain model and a working implementation of every port. Every external system, including the newsroom's fact-check tool and each AI vendor, is a replaceable adapter behind one of those interfaces.

One site, many countries. There is no multi-tenancy. A country is a path segment and a scoping field, not a deployment boundary.

1.2 Scope

In scope:

- Countries, actors (persons, parties, institutions, places), claims, claim updates, documents, categories, claim statuses, citizen submissions, media
- Multi-format, multi-source ingestion with AI-assisted extraction and mandatory human review
- A pluggable verification backend, with Superdesk and a native in-admin reviewer
- Public site: country and actor pages, filterable claim listings, claim pages with update timeline and map, full-text search
- Embeddable widgets, a read-only public JSON API, CSV export
- English and French content locales

Out of scope, with the data model left able to accommodate each later:

Excluded	Left open how
Per-country editorial scoping	`User.countries` exists as an unenforced relationship; enabling it is a `where` clause and a field guard, with no schema change
Custom domains per country	Routing resolves country from the path, so a domain-to-country map is additive
Per-country theming	Branding fields live on the country record
Public user accounts, bookmarking, alerts	Submissions are anonymous with optional contact details
Real-time collaborative editing	Payload document locking is sufficient
Automated claim detection from news	The `SourceConnector` port would host it
Crowdsourced verification	Submissions carry evidence; an accountable fact-checker decides
Elasticsearch or Algolia	One module isolates the search layer

1.3 Technology stack

Layer	Choice
Framework	Next.js (App Router), React 19, TypeScript strict
CMS and admin	Payload CMS 3.x
Database	MongoDB (`@payloadcms/db-mongodb`)
Object storage	S3-compatible
Text extraction	Apache Tika, self-hosted
AI	Vercel AI SDK behind an `LLMProvider` port
Jobs	Payload Jobs, with an atomic workflow lock
Errors and tracing	Sentry

Layer	Choice
Tests	Vitest (integration), Playwright (end to end)
Runtime	Node `^18.20.2 || >=20.9.0`, pnpm `^9 || ^10`

No new infrastructure relative to v2.

1.4 Key constraints

1. **The system must run with no external service.** With only `plugin-core` enabled, and only MongoDB and S3 available, the product is complete and publishable: documents can be uploaded, claims entered, verified, published and served. This configuration is what continuous integration runs against.
2. **No record reaches the public unreviewed.** AI proposes candidates; a human accepts them.
3. **A claim's current status is derived, never entered.** It comes from the most recent published claim update.
4. **The backend performs no outbound request to a user-supplied URL,** except through one explicit connector that runs inside the SSRF guard (§9.4).
5. **Publication state is one writable field plus one computed field.** Ancestry is accounted for in the computed field, so public queries need no joins.
6. **No sync, import or plugin hard-deletes a record.** Upstream deletion maps to an archived state.

1.5 Problems in the v2 system

The v2 codebase cannot run without two third-party services. Airtable holds countries, politicians and source documents and doubles as the operator console, with six pipeline tasks writing human-readable status strings into Airtable cells and `Processed = false` in an Airtable view serving as the work queue. Meedan Check holds published promises, and nothing in the codebase creates a promise record except the Check sync, with `promises.meedanId` required and unique.

Beyond that coupling:

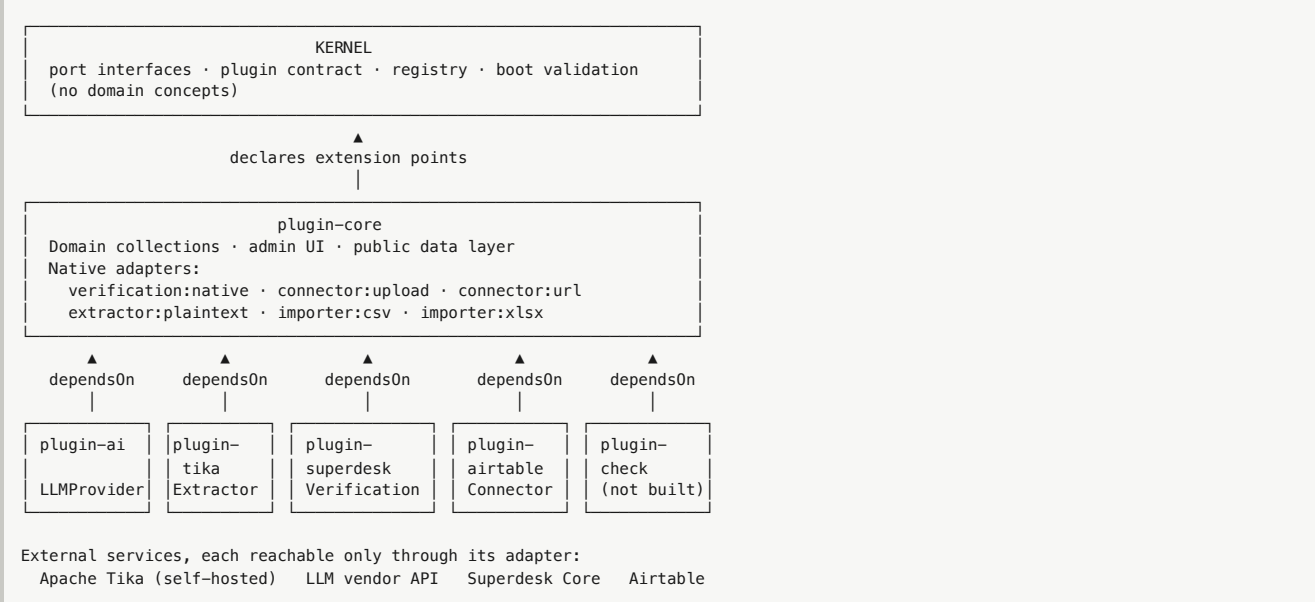
Problem	Evidence in v2
No promise history	`PromiseTimeline` is passed `events={[]}` and renders one synthetic point from `updatedAt`
The public site cannot scale	`force-dynamic` on every route, no caching, `depth: 6` queries, `getAllTenants()` re-run many times per render, and a promise list that loads every published promise unpaginated then filters in the browser
Search is non-functional	`<Search />` renders with no handler, no route and no index
Three inconsistent publication conventions	`tenants.publish` (boolean), `political-entities.publish` (boolean), `promises.publishStatus` (free text compared against the literal `"published"` in `src/collections/Promises.ts:12`)
Multi-tenancy is cosmetic	`userHasAccessToAllTenants: () => true` in `src/plugins/index.ts:93`; only 3 of 14 collections carry a `tenant`; `tenant.country` is primary key, display country and subdomain label at once
No entity typing	One `political-entities` collection with free-text `region` and `position`
`category` is free text	The UI offers filtering by it; no taxonomy collection exists
Accented text is destroyed	`cleanText` in `src/tasks/extractDocuments.ts:19` strips `[^\x00-\x7F]` on a bilingual English and French product
Pages cannot be embedded	`frame-ancestors 'self'` in `src/lib/security/headers.mjs:53` with no embed route exception

The product has never been deployed to production and holds no production data, so v3 carries no migration obligation: no field mapping, no URL history, no cutover.

---

2. System architecture

2.1 High-level architecture



2.2 Layer responsibilities

Kernel. Port interfaces, the plugin contract, the plugin registry, and startup validation. Contains no domain concepts and could host a different product.

plugin-core. Every domain collection and global, the admin experience, the public data layer, and a working implementation of every port. Enabling only core yields a complete, publishable system whose sole external dependencies are MongoDB and S3.

Integration plugins. Each declares `dependsOn: ["core"]`, extends core's collections through published extension points, and registers one or more port implementations. A plugin never imports from another plugin and never reads `process.env` directly.

2.3 Operating modes

`PT_PLUGINS` selects the enabled set. Every row below is a supported configuration.

PT_PLUGINS	Ingestion	Text extraction	Candidate generation	Verification
`core`	Admin upload, URL, CSV and XLSX import	Plain text and HTML	None; claims entered by hand	Review queue in the admin
`core,tika`	as above	PDF, DOCX, XLSX, PPTX, ODF, RTF and more	None	Admin
`core,tika,ai`	as above	as above	AI candidates into the review queue	Admin
`core,tika,ai,superdesk`	as above	as above	AI candidates into the review queue	Superdesk
`core,tika,ai,superdesk,airtable`	plus Airtable	as above	as above	Superdesk

2.4 Public routing and URL structure

One origin. The first path segment is the country code, and each actor kind has its own typed segment.

```
/                               Global home, country selector
/about, /methodology, /{pageSlug}   Global pages

/{cc}                           Country landing
/{cc}/politicians               Listing, filterable
/{cc}/politicians/{slug}        Person profile
/{cc}/parties                   Listing
/{cc}/parties/{slug}            Party profile
/{cc}/institutions              Listing
/{cc}/institutions/{slug}       Institution profile
/{cc}/places                    Listing
/{cc}/places/{slug}             Place profile
/{cc}/promises                  All claims, filterable and paginated
/{cc}/promises/{slug}           Claim page
/{cc}/search                    Search results
/{cc}/{pageSlug}                Country page
```

`cc` is `Country.code`, the lowercase ISO 3166-1 alpha-2 code.

Actor kind	URL segment
`person`	`politicians`
`party`	`parties`
`institution`	`institutions`
`place`	`places`

Typed segments mean a slug need only be unique within `(country, kind)`, so `/ke/politicians/nairobi` and `/ke/places/nairobi` can coexist. A flat `/{cc}/{slug}` would force global slug uniqueness and require a lookup across every kind before the server knows what it is rendering, which is why v2's catch-all route runs to 443 lines with four near-identical branches.

Slug validation on `Pages` reserves these first segments under a country: `politicians`, `parties`, `institutions`, `places`, `promises`, `search`, `api`, `embed`, `sitemap.xml`, `feed`.

Middleware resolves the country once per request from a cached code-to-id map, invalidated on country change, and passes it down the request context. Components receive the resolved country and actor and never re-query by slug. An unknown country code returns 404 with no fallback.

---

3. Plugin architecture

3.1 Overview

Plugins are Payload CMS plugins: functions that receive the incoming Payload config and return a modified config. `PT_PLUGINS` enables them, and the value is evaluated at boot, so toggling a plugin requires a restart. This is deliberate. Field injection and route registration cannot be changed per request, and a mid-session change of ownership would create the write conflicts §3.6 exists to prevent.

Two kinds of plugin exist, and the distinction drives §3.5:

- **Optional capability.** Disabling it removes a feature. `plugin-tika` and `plugin-ai` are of this kind.
- **Alternative implementation of a required capability.** Exactly one must be active. `plugin-core`'s native reviewer and `plugin-superdesk` are of this kind.

3.2 Ports and cardinality

A port is a TypeScript interface in the kernel. An adapter implements it and registers under a plugin-qualified id.

Port	Cardinality	Question it answers	Core adapter	Other adapters
`VerificationProvider`	exactly one	Where do humans verify claims?	`native`	`superdesk`, `check` (specified, not built)
`SourceConnector`	zero or more, `upload` always on	Where does content come from?	`upload`, `url`	`airtable`, `sheets`, `feed`
`ContentExtractor`	one or more, selected by MIME type	How do bytes become text?	`plaintext`	`tika`, `ocr`
`StructuredImporter`	zero or more	How do tabular rows become records?	`csv`, `xlsx`	`sheets`
`LLMProvider`	many registered, one selected	Which model proposes candidates?	none	`google`, `openai`, `anthropic` and others

Ingestion is three ports rather than one because the two shapes of input are different problems. A CSV of claims a partner has already compiled is a row-to-record mapping that needs no language model. A 200-page manifesto is text extraction followed by AI extraction. v2 had one path and assumed PDFs.

3.3 Port interfaces

```
// kernel/ports.ts

export interface VerificationProvider {
  readonly id: string
  /** Send a claim for human verification. Returns the external reference, if any. */
  submit(claim: ClaimRef, ctx: PortContext): Promise<ExternalRef | null>
  /** Pull verdicts changed since a cursor. Used for reconciliation and backfill. */
  pull(since: SyncCursor, ctx: PortContext): Promise<VerdictUpdate[]>
  /** Handle an inbound push from the provider, if it supports one. */
  receive?(payload: unknown, headers: Headers, ctx: PortContext): Promise<VerdictUpdate[]>
  health(): Promise<HealthResult>
}

export interface SourceConnector {
  readonly id: string
  readonly mode: 'push' | 'pull' | 'manual'
  /** Pull mode only: list items awaiting ingest. */
  list?(since: SyncCursor, ctx: PortContext): Promise<SourceItem[]>
  fetch(item: SourceItem, ctx: PortContext): Promise<FetchedContent>
  health(): Promise<HealthResult>
}

export interface ContentExtractor {
  readonly id: string
  /** MIME types this extractor claims; most specific wins, '*' is the fallback. */
  readonly accepts: readonly string[]
  extract(input: FetchedContent, ctx: PortContext): Promise<ExtractedText>
  health(): Promise<HealthResult>
}

export interface StructuredImporter {
  readonly id: string
  readonly accepts: readonly string[]
  /** Inspect without writing: used to build and preview a column mapping. */
  inspect(input: FetchedContent): Promise<TableShape>
  import(input: FetchedContent, mapping: ImportMapping, ctx: PortContext): Promise<ImportResult>
}

export interface LLMProvider {
  readonly id: string
  readonly models: readonly ModelDescriptor[]
  generate(req: GenerateRequest, ctx: PortContext): Promise<GenerateResult>
  health(): Promise<HealthResult>
}
```

`PortContext` carries the Payload instance, a scoped logger, an `AbortSignal` and the resolved settings group for that plugin.

3.4 Plugin manifest

```
export interface PluginManifest {
  readonly id: string                  // 'core' | 'superdesk' | 'ai' | ...
  readonly version: string
  readonly dependsOn?: readonly string[]

  /** Collections and globals this plugin owns outright. */
  collections?: CollectionConfig[]
  globals?: GlobalConfig[]

  /** Changes to collections owned by another plugin. */
  extend?: Record<string, CollectionExtension>

  /** Port implementations, keyed by port name. */
  provides?: {
    verification?: VerificationProvider
    connectors?: SourceConnector[]
    extractors?: ContentExtractor[]
    importers?: StructuredImporter[]
    llm?: LLMProvider[]
  }

  tasks?: TaskConfig[]
  workflows?: WorkflowConfig[]
  /** Declared, not registered: App Router routes are filesystem entries. */
  routes?: PluginRoute[]
  /** A group appended to the Settings global, namespaced by plugin id. */
  settings?: Field[]
  /** Validated at boot when the plugin is enabled. */
  requiredEnv?: readonly string[]
}

export interface CollectionExtension {
  /** Appended fields. Always optional, see §3.6. */
  addFields?: Field[]
  /** Core field names this plugin becomes the authority for. */
  ownFields?: string[]
  /** Hide create and delete: this plugin owns the record lifecycle. */
  ownLifecycle?: boolean
  /** A banner shown on the edit view explaining who owns this data. */
  notice?: Record<string, string>
  hooks?: Partial<CollectionHooks>
}
```

Configuration:

```
PT_PLUGINS=core,tika,ai,superdesk       # core is implied but may be listed
PT_VERIFICATION=superdesk               # which VerificationProvider wins
PT_LLM_MODEL=google:gemini-2.5-pro      # which LLMProvider and model
```

3.5 Boot sequence and port resolution

1. Read `PT_PLUGINS` and load each manifest.
2. Verify `dependsOn` is satisfied, then sort topologically. A cycle is a fatal error.
3. Resolve the selected adapter for each exclusive port. Zero or more than one is a fatal error, and the message names the candidates and the environment variable that selects between them.
4. Merge collections, then apply `extend` in dependency order.
5. Append each plugin's settings group to the Settings global under its id.
6. Validate `requiredEnv` for enabled plugins only.
7. Register tasks and workflows.
8. Log the resolved configuration as one structured line: plugins, port bindings, adapter versions.

Failing at boot replaces the v2 reference architecture's `requirePlugin()` pattern, where a disabled plugin's route returned `503` at request time. That behaviour suits a genuinely optional endpoint and remains in use for plugin routes (§3.10). It is wrong for a required capability with interchangeable implementations: a deployment with no verification provider is misconfigured and must not start.

3.6 Field ownership and locking

When a plugin owns data, exactly one writer exists. Six rules, enforced by the kernel and by code review.

1. **Injected fields are always optional** and carry a sensible default. A plugin may be removed after months of use, and required fields would invalidate every existing record.
2. **A plugin may not change a core field's type,** only its admin presentation and access.
3. **Owned fields are read-only in the admin** and reject writes from anyone but the owning plugin and a super-admin, whose override is audit-logged.
4. **Lifecycle ownership hides create and delete** for that collection and shows a notice explaining where the data comes from.
5. **Plugin fields are namespaced** under a group named for the plugin, so `superdesk.guid` rather than a bare `guid`. Two plugins cannot collide, and removal is a clean sweep.
6. **A plugin never deletes a core record.** Upstream deletion maps to `recordStatus: archived`.

Example: `plugin-superdesk` taking ownership of verification.

```
extend: {
  claims: {
    addFields: [
      {
        name: 'superdesk',
        type: 'group',
        admin: { readOnly: true, condition: () => true },
        fields: [
          { name: 'guid', type: 'text', index: true },
          { name: 'version', type: 'number' },
          { name: 'state', type: 'text' },
          { name: 'itemUrl', type: 'text' },
          { name: 'lastSyncedAt', type: 'date' },
          { name: 'lastSyncStatus', type: 'select', options: ['success', 'failed', 'pending'] },
          { name: 'lastSyncError', type: 'text' },
        ],
      },
    ],
    ownFields: ['verification.verdict', 'verification.verifiedAt', 'verification.rationale'],
    notice: {
      en: 'Verification for this claim is managed in Superdesk. Changes made here will be overwritten on the next sync.',
      fr: 'La vérification de cette affirmation est gérée dans Superdesk. Les modifications effectuées ici seront écrasées lors de la
prochaine synchronisation.',
    },
  },
}
```

The adapter does not own title, statement, category, promisor or the update timeline. Those stay editable in PromiseTracker whichever verification backend is active. Only the verdict crosses the boundary.

When a plugin owns a capability, the matching in-CMS role loses it, so two sources of truth never exist.

Active plugin	Capability it takes	Effect
`plugin-superdesk`	`verification.verdict`	The `reviewer` role can no longer set a verdict in the admin. Candidate triage stays in PromiseTracker
`plugin-airtable`	Document and actor intake	`contributor` creation of those records is disabled

3.7 Plugin catalogue

3.7.1 plugin-core

Implements `VerificationProvider:native`, `SourceConnector:upload`, `SourceConnector:url`, `ContentExtractor:plaintext`, `StructuredImporter:csv` and `StructuredImporter:xlsx`. Owns every collection and global in §4.

The native verification provider is a null implementation. With it selected, `verification.verdict` is an ordinary editable field and no synchronisation exists. A saved admin view, "Awaiting verification", is the queue; a reviewer sees the statement, the source document with the quote highlighted, the actor and the extraction provenance.

```
const nativeVerification: VerificationProvider = {
  id: 'native',
  async submit() { return null },        // nothing leaves the system
  async pull() { return [] },            // nothing comes back
  async health() { return { healthy: true, latencyMs: 0 } },
}
```

3.7.2 plugin-superdesk

Implements `VerificationProvider:superdesk`. Owns `verification.verdict`, `verification.verifiedAt` and `verification.rationale` on `claims`, and injects a read-only `superdesk` group.

Superdesk Core is Sourcefabric's newsroom production system. It has no native fact-check or claim model, so the adapter builds one from a content profile and custom vocabularies. The newsroom must configure its instance before the integration can work, and the items marked **CONFIRM** below are instance-specific.

The three APIs.

API	Prefix	Methods	Use
Internal	`/api` (`URL_PREFIX`)	full CRUD	The only write path
Production API	`/prodapi/v1` (`PRODAPI_URL_PREFIX`)	GET only	Reading items in any state, including unpublished
Content API	deployment-specific	GET only	Published items, NINJS-shaped

CONFIRM the Content API prefix. It derives from `CONTENTAPI_URL`; Sourcefabric's own compose file yields `/capi/items` while other deployments yield `/v1/items`. Do not hardcode. **CONFIRM** also whether the Production API is deployed, since it is a separate process many installations omit.

Authentication.

API	Mechanism
Internal	`POST /api/auth_db` with service-account credentials returns a session token, sent as `Authorization: Basic base64("<token>:")`. Sessions expire after `SESSION_EXPIRY_MINUTES`, default 240; the adapter re-authenticates on 401
Production API	OAuth2 client credentials for a JWT: `POST /api/auth_server/token` with `grant_type=client_credentials`, then `Authorization: Bearer <jwt>`, scope `ARCHIVE_READ`. Registered with `manage.py auth_server:register_client`
Content API	A subscriber token, the `_id` of a `subscriber_token` document created via `POST /api/subscriber_token`

The auth server grants read-only scopes and has no write scope, so creating items requires a real Superdesk user account with the `archive` privilege.

Claim representation. A content item is a flat JSON document close to IPTC NINJS.

PromiseTracker	Superdesk	Notes
Claim identity	`guid`	Superdesk-generated, stored back as `superdesk.guid`
`title`	`headline`	
`statement`	`body_html`	Verbatim quote in a blockquote, plus context
`summary`	`abstract`	
`type`	`subject[]`, `scheme: 'claim_type'`	Custom vocabulary
`category`	`subject[]`, `scheme: 'claim_category'`	Custom vocabulary
`verification.verdict`	`subject[]`, `scheme: 'verdict'`	Custom vocabulary. **CONFIRM** the scheme id
`verification.rationale`	`body_html` of the published fact-check	
Claim id back-reference	`extra.promisetracker_id`	Extra field, `field_type: 'text'`
Claim URL	`extra.promisetracker_url`	Extra field
Language	`language`	
Item kind	`profile`	A content profile such as `fact_check`. **CONFIRM** the id

Anything the adapter must query goes in `subject` with a scheme; anything it only carries goes in `extra`. A vocabulary with `field_type: null` plus a `service` writes into `subject` as `{ qcode, name, scheme }`, survives into NINJS and is queryable in Elasticsearch. A vocabulary with `field_type` set becomes an extra field at `extra.<cv_id>`, and the `extra` dict is mapped `dynamic: false`, so filtering on it may not work without a mapping change. **CONFIRM** whether `extra.*` is mapped on the target instance.

CONFIRM the vocabulary ids and `qcode` values, the content profile id, and the desk and stage ids. The adapter validates all of them at boot and refuses to start on a mismatch.

Outbound.

```
POST /api/archive
Authorization: Basic base64("<session-token>:")
Content-Type: application/json

{
  "type": "text",
  "profile": "<fact_check profile id>",
  "headline": "Ruto: 'We will build 200 new health centres by 2027'",
  "body_html": "<blockquote>verbatim statement</blockquote><p>Source: 2022 manifesto, p.41</p>",
  "abstract": "...",
  "language": "en",
  "subject": [
    { "qcode": "unverified",       "name": "Unverified",       "scheme": "verdict" },
    { "qcode": "campaign-promise", "name": "Campaign promise", "scheme": "claim_type" },
    { "qcode": "health",           "name": "Health",           "scheme": "claim_category" }
  ],
  "extra": {
    "promisetracker_id":  "6745abc",
    "promisetracker_url": "https://promisetracker.org/ke/promises/200-health-centres"
  },
  "task": { "desk": "<desk id>", "stage": "<stage id>" }
}
```

Superdesk fills `guid`, `_id`, `family_id`, `unique_id`, `version` and `original_creator`, and sets `state: 'draft'`. `task.desk` and `task.stage` are required in practice: without them the item lands in the service account's personal workspace, where no fact-checker will see it.

Post to `/api/archive`, not `ingest`. Archive places the item inside the newsroom at `state: 'draft'`. A fetched ingest item receives a new `guid`, which breaks the stable identifier the adapter depends on.

Inbound, primary path. A Superdesk subscriber with an `http_push` destination.

```
{
  "name": "PromiseTracker",
  "is_active": true,
  "subscriber_type": "all",
  "destinations": [{
    "name": "promisetracker",
    "format": "ninjs",
    "delivery_type": "http_push",
    "config": {
      "resource_url": "https://promisetracker.org/api/plugins/superdesk/items",
      "assets_url":   "https://promisetracker.org/api/plugins/superdesk/assets",
      "secret_token": "<shared secret>"
    }
  }],
  "products": ["<product id>"]
}
```

Superdesk signs each push:

```
x-superdesk-signature: sha1=<hex HMAC-SHA1(secret_token, raw_body)>
```

The handler verifies this signature in constant time before parsing the body. HMAC-SHA1 is weak by modern standards but is what Superdesk implements; the secret's entropy compensates, and the endpoint is treated as untrusted input regardless. **CONFIRM** that `http_push` is available and that a subscriber and product can be configured.

Inbound, secondary path. A reconciliation poll every 15 minutes. Pushes are not guaranteed: a deploy, a network fault or a disabled subscriber loses them silently.

```
GET {contentApiPrefix}/items
      ?filter={"range":{"versioncreated":{"gte":"2026-11-06T10:00:00+0000"}}}
      &sort=[("versioncreated",1)]
      &max_results=200
Authorization: Basic base64("<subscriber-token>:")
```

Content API constraints, each of which produces a silently incomplete sync if ignored:

- `start_date` and `end_date` are date-only, inclusive, and cannot be in the future, so they cannot express a 15-minute window. Use `filter` with a raw Elasticsearch range on `versioncreated`.
- With no dates supplied the default window is the last 7 days. Older items are invisible, so a backfill must walk date ranges explicitly.
- `_etag`, `_created` and `_updated` are stripped from Content API responses. Conditional requests are unavailable and the cursor must be `versioncreated`.

- Query parameters are a strict whitelist; anything outside it is an error, not an ignored parameter.
- Pagination defaults to 500 and caps at 1,000.
- Overlap the window by 10 minutes. `versioncreated` is server-side UTC and Elasticsearch has a refresh interval, so an item can become visible after a poll whose window covered it.
- Store the cursor as the maximum `versioncreated` observed, never wall-clock time.
- Date formats differ: `+0000` on the internal and Content APIs, RFC 3339 `+00:00` on the Production API by default.
- Visibility is subscriber-scoped. If the subscriber matches no product, the API returns `200` with an empty list rather than an empty error.

Item state mapping. Superdesk reuses one `guid` across corrections and kills, with a new `version` and `versioncreated`. There is no delete.

Superdesk `state`	**Effect on the claim**
`draft`, `in_progress`, `submitted`	In review, no change
`published`, `corrected`	Apply the verdict and rationale, set `verifiedAt`
`killed`, `recalled`, `takedown`, `unpublished`	Depublish: verdict to `unverified`, `recordStatus` to `hold`, notify an editor
`spiked`	`recordStatus` to `archived`
`scheduled`, `being_corrected`, `correction`	No change; wait for the terminal state

The exact strings matter: it is `in_progress`, not `progress`, and `pubstatus` `withheld` is represented by a constant named `HOLD`.

Deduplicate on `(guid, version)`. A correction republishes the same `guid` at a higher version and is new information; the same `(guid, version)` twice is not. The publish queue retries without ordering guarantees, so apply only when the incoming version exceeds the stored one.

Loop prevention: every inbound item carrying an `extra.promisetracker_id` the system already owns is matched to that claim and never re-created. An inbound item without that field is ignored unless importing newsroom-originated claims is explicitly enabled.

Not used: websocket notifications on port 5100 (resource ids only, no authentication, normally not exposed); `POST /api/webhook` (re-triggers an ingest provider poll, does not receive content); content-list webhooks (present on `develop`, absent from published documentation, **CONFIRM** before relying on them).

Failure behaviour.

Failure	**Behaviour**
Invalid profile, vocabulary, desk or stage id at boot	Refuse to start, naming the bad id
Session expired (401) on write	Re-authenticate once, retry once, then fail the task
Invalid push signature	`401`, log at `warn` with the source IP, do not parse the body
Push for an unknown `guid`	`200` so Superdesk stops retrying, record for reconciliation
Superdesk unreachable	Circuit breaker opens, claims queue locally, health reports `degraded`
Content API empty for 3 consecutive polls	Health `degraded`; a subscriber or product misconfiguration is likely

No part of PromiseTracker stops working when Superdesk is unavailable. Claims are still created, extracted, edited and published; only new verdicts stop arriving.

3.7.3 plugin-check (specified, not built)

Meedan Check is documented as a second implementation of `VerificationProvider` to demonstrate that the interface is not Superdesk-shaped. It is not built in v3.

Port method	**Check equivalent**
`submit`	`createProjectMedia` GraphQL mutation, `media_type: "Claim"`
`pull`	`search(query: { report_status: ["published"] })`
`receive`	`POST /api/plugins/check/webhook`, `annotation_type: "report_design"`

Mapping Check onto the same three verdict fields requires no interface change. The v2 implementation, roughly 2,000 lines across `src/lib/meedan.ts`, `src/lib/syncMeedanReports.ts`, three tasks, a workflow and a webhook route, is not carried forward.

3.7.4 plugin-ai

Implements `LLMProvider` for each configured vendor. Owns the `extractions` collection, the `extractCandidates` task and the review queue UI. Reads `documents.text` and writes `documents.processing.status`; injects no fields.

Adapters wrap the Vercel AI SDK provider factories. The catalogue, model-id resolution and credential validation carry over from `src/lib/ai/providerCatalog.ts` and `providerRegistry.ts`, which already perform this role behind a hardcoded switch. Credentials live in the Settings global, encrypted. One provider is selected for extraction; others may be registered.

Health is a cheap completion against the selected model at startup and on `/api/health`, so exhausted credits surface as `degraded` rather than as a flood of failed documents.

3.7.5 plugin-tika

Implements `ContentExtractor:tika`: a thin client over a self-hosted Apache Tika server. Accepts PDF, Word, Excel, PowerPoint, OpenDocument, RTF, EPUB and the remainder of Tika's format list. Returns NFC-normalised text, a page count, and document metadata as editor suggestions. Two-minute timeout, circuit breaker, participant in `/api/health`.

Tika is a readiness dependency only when enabled, and never for the public read path. In v2 it is an unconditional readiness dependency, so a Tika outage marks the whole application unready even though no public page touches it.

3.7.6 plugin-airtable

Implements `SourceConnector:airtable` and `StructuredImporter:sheets`. Retained as an option for teams still working in Airtable, not as a dependency. With it enabled, Airtable is a source of documents and actors and may own those fields, but it is never the work queue and never the operator console: processing state lives in `Document.processing.status` and is visible in the admin regardless.

3.8 Data governance: recordStatus and _status

`recordStatus` captures intent and is the only publication field a human writes.

Value	Meaning
`ready`	On its own terms, fit to publish
`hold`	Not ready, or temporarily withdrawn. Default for claims and claim updates
`archived`	Retired. Preserved for audit, never public, never auto-deleted

`_status` captures the outcome. It is Payload's draft and published state, computed and never set directly.

```
record._status = 'published'  ⟺  record.recordStatus === 'ready'
                                 ∧ every ancestor's recordStatus === 'ready'
                                 ∧ (claims only) verification.verdict === 'verified'
                else 'draft'
```

Record	Ancestors that must all be `ready`	Extra condition
`Country`	none	none
`Actor`	country	none
`Document`	country	none
`Claim`	country, promisor, source document when set	`verification.verdict === 'verified'`
`ClaimUpdate`	country, promisor, claim	none
`Submission`	never published	none

A claim's source document is an ancestor because the document is the evidence. If a manifesto is withdrawn as inauthentic, every claim extracted from it leaves the site with it.

Two fields rather than one because a single checkbox cannot express "this claim is fit to publish but the document it came from is under review". With two, intent survives the cascade: a claim an editor held individually stays held when its parent is released, and a claim hidden only because its parent was held returns automatically.

The cascade. An `afterChange` hook fires when `recordStatus` changes on any collection.

1. Recompute `_status` for the changed record synchronously, so the save returns correct.

2. Enqueue a background job to walk descendants and recompute each one from its own `recordStatus` plus the refreshed ancestry.
3. On completion, call `revalidateTag` for every affected cache tag (§11.3).

Downward, when a parent moves to `hold` or `archived`, every descendant recomputes to `draft` whatever its own intent. Upward, when a parent returns to `ready`, each descendant recomputes from its own `recordStatus`, so individually held records stay held. A status change never triggers re-extraction, re-verification or other pipeline work.

Because `_status` is always accurate in the database, public queries need no ancestry joins.

```
await payload.find({
  collection: 'claims',
  where: { and: [{ _status: { equals: 'published' } }, { country: { equals: countryId } }] },
})
```

A country with 5,000 claims and 20,000 updates cascades in one background job using batched bulk updates rather than one write per record. The job is idempotent, chunks and checkpoints for large subtrees, and reports counts to Sentry.

Deletion. No sync, import or plugin hard-deletes a record; upstream deletion maps to `recordStatus: archived`. Hard deletion is an explicit admin action on a record the admin has inspected. Deleting a document that has extracted claims, human verification decisions and a published timeline destroys irreversible work, and upstream systems are unreliable about announcing deletions.

Audit.

Event	Recorded where
Any `recordStatus` change	Payload version history, plus a structured log with actor, record, old and new value
`verification.verdict` change	Version history, plus `verifiedBy` and `verifiedAt`
Candidate accept or reject	`Candidate.reviewedBy`, `reviewedAt`, `rejectedReason`
Submission moderation	`moderatedBy`, `moderatedAt`, `moderatorNote`
Super-admin override of a plugin-owned field	Structured log at `warn`, Sentry breadcrumb, before and after values
Plugin sync writes	The plugin's `lastSyncedAt`, `lastSyncStatus`, `lastSyncError` group

Claims and claim updates have versions enabled, so prior published states are recoverable without a separate audit store.

3.9 Plugin development guidelines

1. Never add a required field to a collection another plugin owns. Injected fields are optional.
2. Never change the type of a core field. Modify admin presentation and access only.
3. Namespace every injected field under a group named for the plugin.
4. Read configuration from `PortContext`, never from `process.env` and never from another plugin's settings group.
5. Export route handlers separately. Payload plugins cannot register Next.js routes; follow §3.10.
6. Guard every route handler with an enabled check as its first action.
7. Never hard-delete a core record. Archive instead.
8. Implement `health()` on every adapter, and keep it cheap enough to run on every health poll.
9. A disabled plugin must return the incoming config byte-identical (§8).

3.10 Plugin route convention

App Router routes are filesystem entries and cannot be registered at runtime, so plugin routes follow a convention and are declared in the manifest.

```
src/app/api/plugins/
├── superdesk/
│   ├── items/route.ts      POST, receives pushed NINJS items
│   └── assets/route.ts     GET and POST, media exchange
└── airtable/
    └── webhook/route.ts    POST
```

Every handler's first action is a plugin check: if the plugin is not enabled it returns `503` without executing any application logic. The manifest's `routes[]` declares the path, methods, purpose and required secret for each route, which is how integration tests discover what to exercise rather than hardcoding paths.

4. Data models

`plugin-core` owns everything in §4.1 to §4.6 unless stated. Payload manages `createdAt` and `updatedAt` on every collection; they are not listed. `recordStatus` and the computed `_status` are defined in §3.8.

4.0 Entity relationships



4.1 Core collections

4.1.1 Countries

A country is a routing namespace and a scoping field. One deployment, one database, one set of users.

```
interface Country {
  id: string
  code: string          // ISO 3166-1 alpha-2, lowercase. UNIQUE, INDEXED. The URL segment
  alpha3?: string       // ISO 3166-1 alpha-3, for data interchange
  name: string          // localized
  officialName?: string // localized
  flag?: Media
  defaultLocale: 'en' | 'fr'
  locales: ('en' | 'fr')[]       // which locales this country publishes in
  geo?: GeoRef                   // §4.5
  about?: RichText               // localized
  branding?: { accentColor?: string; heroImage?: Media }
  partners?: Partner[]
  meta?: SeoMeta
  recordStatus: 'ready' | 'hold' | 'archived'   // default 'hold'
}
```

Indexes: `code` unique, `recordStatus`.

`code` is both the identity and the URL segment. Per-country `locales` lets one country publish in English only while another publishes in French only, with no global switch. Archiving a country hides every actor, claim and document beneath it.

4.1.2 Actors

One collection, four kinds. A politician, a party, a ministry and a county all make promises. Modelling them separately would require three further collections, three further route trees and a union query for every "who makes promises here" question.

```
type ActorKind = 'person' | 'party' | 'institution' | 'place'

interface Actor {
  id: string
  country: Country              // required, INDEXED
  kind: ActorKind               // required, INDEXED
  name: string                  // required, useAsTitle
  shortName?: string            // "UDA", "MoH". Used in dense UI
  slug: string                  // unique per (country, kind). INDEXED
  parent?: Actor                // ward → constituency → county; department → ministry
  image?: Media
  description?: RichText        // localized
  links?: { label: string; url: string }[]
  geo?: GeoRef                  // §4.5
  meta?: SeoMeta
  recordStatus: 'ready' | 'hold' | 'archived'   // default 'hold'

  // kind-specific, shown conditionally in the admin

  person?: {
    honorific?: string
    gender?: string
    dateOfBirth?: string
    originPlace?: Actor         // kind='place'. Where they are from
    offices: Office[]           // at least one
  }

  party?: {
    abbreviation?: string
    founded?: string
    colors?: { primary?: string; secondary?: string }
    leader?: Actor              // kind='person'
  }

  institution?: {
    institutionType: 'ministry' | 'agency' | 'county-government' | 'parastatal'
                   | 'commission' | 'legislature' | 'other'
    jurisdiction?: Actor        // kind='place'
    headedBy?: Actor            // kind='person'
    establishedOn?: string
  }

  place?: {
    placeType: 'region' | 'province' | 'state' | 'county' | 'district'
             | 'constituency' | 'ward' | 'city' | 'other'
    adminLevel: number          // 1 = first-level subdivision, 2 = second
    population?: number
    externalCode?: string       // GADM / geoBoundaries / HASC identifier
  }
}

interface Office {
  title: string                 // "President", "Cabinet Secretary for Health"
  institution?: Actor           // kind='institution'
  jurisdiction?: Actor          // kind='place'
  party?: Actor                 // kind='party', at the time of this office
  from: string                  // required
  to?: string                   // open-ended while in office
  isCurrent: boolean            // derived from `to`, stored for querying, INDEXED
}
```

Indexes: `(country, kind, slug)` unique, `(country, kind, recordStatus)`, `parent`, `person.offices.isCurrent`.

Slug uniqueness is scoped to `(country, kind)`, which the typed URL segments in §2.4 keep apart. `offices[]` replaces v2's flat `position`, `region`, `periodFrom` and `periodTo`, which could represent only one office and lost a person's history on a change of job. `originPlace` drives "where this person is from" and `offices[].jurisdiction` drives "where they govern"; the two are frequently different.

4.1.3 Claims

```
type ClaimType = 'campaign-promise' | 'policy-pledge' | 'budget-commitment' | 'public-statement'

interface Claim {
  id: string
  country: Country              // required, INDEXED. Denormalised from promisor
  type: ClaimType               // required, INDEXED. default 'campaign-promise'
  title: string                 // required, localized. Editorial headline
  slug: string                  // unique per country. INDEXED
  statement: string             // required. The exact words, verbatim, never paraphrased
  summary?: RichText            // localized. Editorial context

  promisor: Actor               // required, INDEXED
  coPromisors?: Actor[]
  jurisdiction?: Actor          // kind='place', if narrower than the country

  category: Category            // required, INDEXED
  tags?: string[]

  source?: Document             // INDEXED. Absent for a manually entered claim
  sourceLocator?: {
    page?: number
    chunk?: number
    timestamp?: string          // for audio and video sources
    excerptStart?: number
  }
  sourceUrl?: string            // attribution only. NEVER fetched by the backend (§9.4)
  madeOn?: string
  targetDate?: string           // the deadline the claim set itself, if any
  language: 'en' | 'fr'

  // tracking
  currentStatus?: ClaimStatus   // READ-ONLY. Derived from the latest published ClaimUpdate
  statusAsOf?: string           // READ-ONLY
  updateCount?: number          // READ-ONLY. Denormalised for listings

  // verification
  verification: {
    verdict: 'unverified' | 'verified' | 'false' | 'misleading'   // default 'unverified'
    verifiedAt?: string
    verifiedBy?: User | string
    rationale?: RichText
    report?: Media
  }

  // provenance
  extraction?: Extraction       // set when AI-derived
  confidence?: number           // 0 to 1
  enteredBy: 'ai' | 'manual' | 'import'   // INDEXED

  image?: Media
  meta?: SeoMeta
  recordStatus: 'ready' | 'hold' | 'archived'   // default 'hold'
}
```

Indexes: `(country, recordStatus, _status)`, `(promisor, _status)`, `(country, category, _status)`, `(country, currentStatus)`, `slug` unique per country, `verification.verdict`, and a text index on `title`, `statement` and `summary`.

Versions and drafts are enabled, so every published change is recoverable and an editor can stage a correction without taking the page down.

No human writes `currentStatus`. An `afterChange` hook on `claim-updates` recomputes it. `statement` is the verbatim quote and is what the quote-verification guard (§5.4) checks against the source document. `enteredBy` makes extraction quality an answerable question. `verification.verdict` is the only field a verification plugin owns.

4.1.4 Claim statuses

Authored in the CMS. In v2 this collection was seeded from Meedan's `verification_statuses`, joined on `meedanId` in three places, and only ever created, never updated, so a label corrected upstream never reached the site.

```
interface ClaimStatus {
  id: string
  slug: string                  // unique, stable: 'kept', 'in-progress', 'broken'
  label: string                 // required, localized
  description: string           // required, localized. Public explanatory copy
  group: 'not-started' | 'in-progress' | 'kept' | 'broken' | 'inconclusive'   // required
  color: string                 // hex, used for chips and charts
  textColor: string             // hex, default '#ffffff'
  order: number
  isDefault?: boolean           // suggested status for a new claim's first update
}
```

`group` drives every scorecard, chart and filter. v2 configured chart groups per page by picking statuses into three `chartGroups` on a `hero` block, with a hardcoded fallback and a balancing redistribution when the configuration was absent. Grouping belongs on the status record, defined once.

Statuses are global rather than per country, which keeps cross-country comparison meaningful.

4.1.5 Claim updates

The dated history that makes the system a tracker.

```
interface ClaimUpdate {
  id: string
  claim: Claim                  // required, INDEXED
  date: string                  // required, INDEXED. When the development happened,
                                // not when it was recorded
  status: ClaimStatus           // required. The status as of this date
  title?: string                // localized
  narrative: RichText           // required, localized. What happened, and how we know
  evidence: Evidence[]          // at least one for a status change
  author?: User
  attribution?: string          // credit for a citizen contributor, with consent
  sourceType: 'editorial' | 'citizen-submission' | 'import' | 'automated'
  submission?: Submission       // set when promoted from the moderation queue
  recordStatus: 'ready' | 'hold' | 'archived'   // default 'hold'
}

interface Evidence {
  label: string                 // localized
  kind: 'link' | 'document' | 'media' | 'quote'
  url?: string                  // attribution only, never fetched
  document?: Document
  media?: Media
  quote?: string
  publisher?: string
  publishedOn?: string
}
```

Indexes: `(claim, date)` descending for the timeline query, `(claim, recordStatus)`.

On create, update or delete of a published claim update, a hook sets three fields on the parent claim.

```
claim.currentStatus = status of the most recent ClaimUpdate where _status === 'published',
                      ordered by date desc, then createdAt desc
claim.statusAsOf    = that update's date
claim.updateCount   = count of published updates
```

With no published updates `currentStatus` is null and the public page shows the claim as not yet assessed. A claim with a verdict but no updates is recorded but not yet tracked, which is a legitimate state.

`date` is separate from `createdAt` because claims are tracked retrospectively: an editor in 2026 records that a hospital opened in 2024. The timeline orders by when events happened.

4.1.6 Categories

```
interface Category {
  id: string
  slug: string                  // unique, INDEXED
  name: string                  // required, localized
  description?: string          // localized
  icon?: Media
  color?: string
  parent?: Category             // one level of nesting, no deeper
  order: number
}
```

Global rather than per country, so a cross-country category query needs no reconciliation.

4.1.7 Documents

Source material. One explicit processing state replaces v2's state spread across `files exists`, `extractedText.text exists`, `fullyProcessed`, `extractionReview` and a status string in an Airtable cell.

```
type ProcessingStatus =
  | 'pending' | 'fetching' | 'fetched'
  | 'extracting' | 'extracted'
  | 'analysing' | 'analysed'
  | 'imported'          // structured-import path, §5.5
  | 'completed' | 'failed' | 'needs-review'

interface Document {
  id: string
  country: Country              // required, INDEXED
  title: string                 // required
  actors?: Actor[]              // who this document is by or about
  type: 'manifesto' | 'speech' | 'budget' | 'policy' | 'report'
      | 'news' | 'dataset' | 'other'
  language: 'en' | 'fr'
  publishedOn?: string
  publisher?: string

  file?: Media                  // the stored artefact
  sourceUrl?: string            // attribution. Fetched ONLY via the `url` connector
  connector: string             // which SourceConnector brought it in
  mimeType?: string             // detected, not trusted from the client

  processing: {
    status: ProcessingStatus    // required, INDEXED. default 'pending'
    stage?: string              // label for the current operation
    message?: string            // last error or notice, shown in the admin
    extractorId?: string
    startedAt?: string
    lastAttemptAt?: string
    completedAt?: string
    retryCount: number          // default 0. One budget across errors and stuck sweeps
  }

  text?: string                 // extracted plain text, Unicode-normalised (NFC)
  textChars?: number
  pageCount?: number

  recordStatus: 'ready' | 'hold' | 'archived'   // default 'ready'
}
```

Indexes: `(processing.status, processing.lastAttemptAt)` for the work queue and stuck sweep, `(country, recordStatus)`, `actors`.

`processing.status` is the work queue. Making it a first-class indexed enum decouples the pipeline from any ingestion source. Documents are not public: claim pages link to them as evidence by URL, but the collection is admin-read-only.

4.1.8 Submissions

Citizen input, replacing v2's embedded Airtable form whose rows nothing read back.

```
interface Submission {
  id: string
  claim?: Claim                 // INDEXED. Absent for "a promise is missing" reports
  country?: Country
  kind: 'update' | 'correction' | 'new-claim'
  body: string                  // required
  evidenceUrl?: string          // attribution only. NEVER fetched
  attachment?: Media
  submitterName?: string
  submitterEmail?: string
  consentToAttribute: boolean   // default false
  status: 'new' | 'accepted' | 'rejected' | 'spam'   // INDEXED. default 'new'
  moderatorNote?: string
  moderatedBy?: User
  moderatedAt?: string
  claimUpdate?: ClaimUpdate     // set when promoted
  ipHash?: string               // salted hash, never the raw address
  userAgent?: string
}
```

Access: `create` is public and rate-limited, `read` is authenticated only. A submission is never rendered publicly in any status.

4.2 Plugin collections

4.2.1 Extractions (`plugin-ai`)

The AI audit trail and the review queue's backing store.

```
interface Extraction {
  id: string
  document: Document            // required, INDEXED
  model: string                 // 'google:gemini-2.5-pro'
  promptVersion: string         // so a prompt change is attributable
  status: 'pending' | 'completed' | 'failed' | 'partial'
  error?: string
  tokensUsed?: number
  durationMs?: number
  rawResponse?: JSON            // retained for debugging, see retention in §9.10
  candidates: Candidate[]
}

interface Candidate {
  uniqueId: string              // stable, generated, survives re-extraction
  category?: string             // free text as proposed, mapped to a Category on acceptance
  summary: string               // the proposed claim title
  quotes: string[]              // verbatim spans, each verified against document.text
  confidence?: number
  decision: 'pending' | 'accepted' | 'rejected'   // INDEXED
  claim?: Claim                 // set on acceptance
  rejectedReason?: string
  reviewedBy?: User
  reviewedAt?: string
}
```

Re-extracting a document creates a new `Extraction` and leaves prior ones intact, so decisions already made survive and extraction quality over time is measurable.

4.3 Supporting collections

Media. `alt` (required), `externalUrl` , `checksum` (SHA-256, indexed, drives deduplication), S3-backed, public read.

Users. `roles` (hasMany; see §9.7), `name` , and `countries?` , a relationship that exists but is not enforced in v3.

Pages. One collection. An optional `country` relationship, where null means a global page (`/about`) and a value means a country page (`/ke/about`). The same block set serves both, and blocks requiring a country render empty when there is none.

Partners. `name` , `description` , `image` , `url` , optional `countries[]` .

WorkflowLocks. System collection for the atomic cron guard, carried over verbatim from `src/collections/WorkflowLocks.ts` .

ImportJobs. One record per structured import: the file, the chosen `StructuredImporter` , the column mapping, a dry-run preview, counts of created, updated and skipped rows, and per-row errors.

4.4 Globals

Site settings. Site title and description, logos, navigation, footer, legal links, social links, newsletter configuration, default SEO. Country-specific overrides live on the `Country` record, not in a second global.

Settings. Integration credentials, super-admin read and write only. Core contributes feature toggles; each plugin appends a namespaced group.

```
interface Settings {
  features: {
    aiExtractionEnabled: boolean
    submissionsEnabled: boolean
    publicApiEnabled: boolean
    embedsEnabled: boolean
  }
  // appended by plugins, namespaced by plugin id:
  superdesk?: { baseUrl; clientId; clientSecret; deskId; stageId; profileId; pushSecret }
  ai?: { credentials[]; model; limits }
  tika?: { url }
  airtable?: { apiKey; baseId }
}
```

Credentials live in the database because they change per deployment without a redeploy, and because `src/fields/encryptedSecret.ts` already encrypts them at rest and masks them on read. Environment variables select which plugins run; the database holds what they need to run.

4.5 Shared field groups

```
interface GeoRef {
  centroid?: { lat: number; lng: number }
  bounds?: { north: number; south: number; east: number; west: number }
  boundary?: Media              // a GeoJSON file, stored like any other upload
  boundarySource?: string       // 'geoBoundaries', 'GADM', 'OSM'. Attribution
  boundaryLicense?: string      // required when boundary is set
  externalCode?: string         // the id in that source, for refresh
  defaultZoom?: number
}
```

Boundary polygons are stored GeoJSON uploads rather than live third-party geometry. Boundary datasets carry licence terms and change rarely, and a live tile or geometry dependency would reintroduce the coupling this architecture removes. Attribution is required when a boundary is present. Boundary files can be large: the admin warns above 2 MB and the public site serves a simplified rendition. **CONFIRM** the topology-preserving simplification step at implementation time.

`SeoMeta` carries `title`, `description` and `image` overrides, all optional, all localized.

4.6 Localization

Content locales are `en` and `fr`, driven by `NEXT_PUBLIC_LOCALES`.

Localized: `Country.name`, `officialName`, `about`; `Actor.name`, `description`; `Claim.title`, `summary`; `ClaimUpdate.title`, `narrative`; `Evidence.label`; `ClaimStatus.label`, `description`; `Category.name`, `description`; all page block text; site settings text.

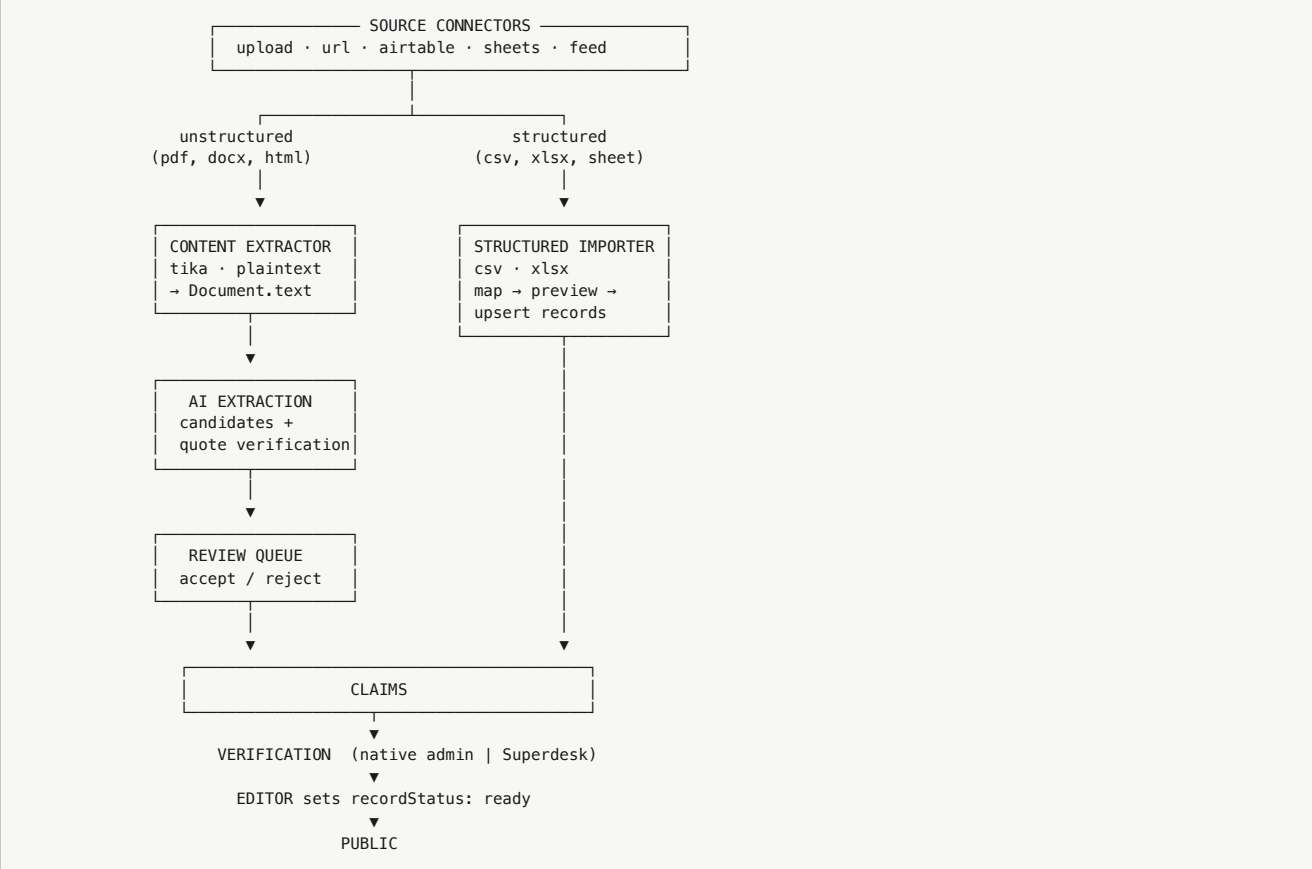
Not localized: slugs, `Claim.statement`, codes, dates, colours, numbers. A verbatim quote is not translated; a translation would be a different object.

4.7 Index summary

Collection	Indexes
`countries`	`code` unique, `recordStatus`
`actors`	`(country, kind, slug)` unique, `(country, kind, recordStatus)`, `parent`, `person.offices.isCurrent`
`claims`	`(country, recordStatus, _status)`, `(promisor, _status)`, `(country, category, _status)`, `(country, currentStatus)`, `(country, slug)` unique, `verification.verdict`, text on `title` / `statement` / `summary`
`claim-updates`	`(claim, date)` desc, `(claim, recordStatus)`
`claim-statuses`	`slug` unique, `group`, `order`
`categories`	`slug` unique, `parent`, `order`
`documents`	`(processing.status, processing.lastAttemptAt)`, `(country, recordStatus)`, `actors`
`extractions`	`document`, `candidates.decision`
`submissions`	`status`, `claim`, `ipHash`
`media`	`checksum`, `externalUrl`

5. Pipeline architecture

5.1 Ingestion strategy



The structured path bypasses text extraction, AI and the review queue.

Connector	Mode	Plugin	Behaviour
`upload`	manual	core	An editor uploads a file in the admin. Always available
`url`	manual	core	An editor supplies a URL. The backend fetches it through the SSRF guard (§9.4), detects the type and stores the artefact
`airtable`	pull	airtable	Polls a base on a schedule
`sheets`	pull	sheets	Google Sheets as a structured source
`feed`	pull	feed	RSS and Atom monitoring. Future

`Document.sourceUrl` is attribution only and the backend never fetches it. The `url` connector is the single audited exception: an editor explicitly requested that URL, the request passes through the same hardening as every other outbound fetch, and the result is stored as a `Media` artefact so it is never fetched again.

Type detection. The connector reports a MIME type and the pipeline does not trust it. Detection runs in order: magic-byte signature (`src/utils/fileSignature.ts`), then `Content-Type` for the `url` connector, then file extension. A disagreement between signature and declared type marks the document `failed` with a specific message.

Content extraction. MIME type selects the extractor, most specific first.

Extractor	Plugin	Accepts
`plaintext`	core	`text/plain` , `text/markdown` , `text/html` with readability extraction
`tika`	tika	PDF, Word, Excel, PowerPoint, OpenDocument, RTF, EPUB and the remainder of Tika's format list
`ocr`	ocr	Scanned PDFs and images when the text layer is empty. Future

An extractor returns NFC-normalised plain text, a page or section count where known, and recovered document metadata as editor suggestions. Extractors normalise and collapse whitespace and discard no characters.

Spreadsheets are ambiguous: a workbook may be a document to extract claims from or a dataset whose rows are claims. The editor chooses at upload time. The default is the structured path for CSV and the document path for XLSX, and either can be switched before processing starts.

5.2 Document processing pipeline

`Document.processing.status` is the single source of truth and the work queue.

```
pending → fetching → fetched → extracting → extracted → analysing → analysed → completed
   │                                                                              ▲
   └──────────────────── imported (structured path) ──────────────────────────────┘

any state → failed          any state → needs-review
```

Every task writes the status before and after its operation, so a document's position is visible in the admin to editors and to the person who uploaded it.

```
async function runStage<T>(doc: Document, stage: ProcessingStatus, fn: () => Promise<T>) {
  await setStatus(doc.id, stage, { startedAt: new Date() })
  try {
    const result = await fn()
    await setStatus(doc.id, doneStateFor(stage), { completedAt: new Date() })
    return result
  } catch (err) {
    await setStatus(doc.id, 'failed', { message: describe(err), lastAttemptAt: new Date() })
    throw err
  }
}
```

5.3 Claim lifecycle

A claim may originate from AI extraction, manual entry or structured import, recorded in `enteredBy`. Whatever the origin:

1. The claim is created with `recordStatus: 'hold'` and `verification.verdict: 'unverified'`.
2. It is verified through the active `VerificationProvider` (§5.6), which sets the verdict, the rationale and `verifiedAt`.
3. An editor sets `recordStatus: 'ready'`.
4. `_status` becomes `published` only when the verdict is `verified`, the claim's own `recordStatus` is `ready`, and every ancestor is `ready` (§3.8).
5. Claim updates accumulate against it, each with its own `recordStatus`, and the most recent published one determines `currentStatus`.

Verification and publication are separate steps on purpose.

```
verdict = verified           →  the claim is true to the source
recordStatus = ready         →  the editorial content is fit to publish
both, plus ancestry          →  _status = published
```

A verified claim with a half-written summary should not be live, and one switch cannot express that.

Verdicts of `false` and `misleading` are never published and never auto-deleted. They are the audit trail that makes extraction quality and systematic prompt failures visible.

5.4 AI extraction

Owned by `plugin-ai` and optional. With the plugin disabled, documents stop at `extracted` and claims are entered by hand. The algorithm carries over from v2 unchanged in substance.

Pass 1, per-chunk extraction. The text is split into overlapping chunks on paragraph and sentence boundaries. Each chunk goes to the selected `LLMProvider` with a structured-output schema of `{ category, summary, sourceQuotes[] }`. The prompt requires specific verifiable future commitments, forbids merging two commitments into one, and requires exact quotes in the document's own language.

Quote verification. Candidates are deduplicated, then every candidate whose quotes do not literally appear in the normalised document text is discarded. This is `hasVerifiableQuote` and `mergeByAtomicPromise` in `src/tasks/extractPromises.ts:189-213`. It is

the guard against fabricated claims and must survive any refactor.

Pass 2, normalisation. A tool-forced pass splits compound commitments into atomic ones and titles the set, then passes through quote verification again.

Persistence. Results become one `Extraction` with its `candidates[]`, all `decision: 'pending'`. No candidate becomes a claim without a human decision.

Review queue. For one document, each candidate is shown with its proposed title, category and confidence, and with its verified quotes highlighted in the source text. The actions are accept, edit and accept, and reject with a reason. Accepting creates a claim with `enteredBy: 'ai'`, `recordStatus: 'hold'` and `verification.verdict: 'unverified'`, linked to the extraction and candidate. Rejecting records the reason.

Dropped from the v2 implementation:

Dropped	Reason
The 14-provider configuration matrix in the admin	Replaced by `LLMProvider` adapters; a deployment enables the one or two it uses
Per-document cost budgeting (`documentBudget.ts`, `maxCostUsdPerDocument`, per-million-token pricing in settings)	Nine tunables encoding pricing that goes stale. Replaced by one concurrency limit, one per-call timeout and one per-document token ceiling
`ai-extraction-export-rows`	A 28-column denormalised table maintained by hooks on five collections to produce a CSV. Replaced by a live query (§13.1)

5.5 Structured import

For CSV, XLSX and sheets, rows become records directly. The flow is map, preview, commit.

1. **Inspect.** The importer reads headers and the first rows and returns a `TableShape`.
2. **Map.** The editor maps columns to fields. Mappings are saved as reusable profiles.
3. **Preview.** The import runs with no writes and reports rows to create, rows to update, rows to skip, and per-row validation errors with row numbers. An `ImportJob` holds the result.
4. **Commit.** Only after the preview is accepted.

Target shapes are claims (promisor and category resolved by slug or name), actors, claim updates for bulk-loading a history, and documents as metadata plus a source URL.

Each target declares a natural key: `(country, slug)` for claims and actors, `(claim, date)` for updates. Re-importing the same file updates rather than duplicates. An unresolvable reference is reported as a row error rather than creating a stub record.

5.6 Verification

The claim is handed to the active `VerificationProvider`. The outcome is the same three fields in every case: `verification.verdict`, `verifiedAt` and `rationale`. The claim itself stays in PromiseTracker.

5.7 Resilience patterns

5.7.1 Retry policy

Error	Retry	Fallback
Network timeout	3 times, exponential backoff	Next item; alert above a 10% failure rate
Rate limit (429)	Yes, honouring `Retry-After`	Requeue; alert if sustained over 5 minutes
Service unavailable (503)	3 times	Skip; the scheduled sweep retries
Invalid data (400)	No	Mark `failed` with a specific message
Auth error (401, 403)	No	Fail immediately and alert
Server error (500)	2 times	Next item

Backoff starts at 1s, doubles, and caps at 30s.

5.7.2 Timeouts

Operation	Timeout
File fetch	5 min total, 30s per redirect hop
Text extraction	2 min
One AI call	2 min
One document's AI stage	20 min
Verification provider call	1 min
Health check	5s

5.7.3 Stuck-job recovery

A document stuck in a transient state has failed outside the application's control, through a restart, a network stall or a hung dependency.

Transient state	Considered stuck after
`fetching`	10 min
`extracting`	5 min
`analysing`	25 min
any other	2 min

A sweep runs every 5 minutes. `retryCount` is one budget shared by thrown errors and stuck sweeps, so a document cannot alternate between the two indefinitely. Under 5, the sweep re-enqueues at the start of the current stage and increments. At 5, it marks the document `failed` with the last known cause and stops sweeping it.

5.7.4 Circuit breakers

Every external client is wrapped in a breaker: open after 5 consecutive failures, retry after 60s, 3 probe requests before closing. Per-provider concurrency and minimum-interval limits are configured per adapter rather than globally.

5.7.5 Idempotency

- **Inbound pushes** carry a delivery id, checked against the last-seen id on the target record before processing. v2's `/api/meedan-sync` implements this with a SHA-256 body fingerprint and an in-flight 409; the mechanism carries over.
- **Record creation** relies on database unique indexes rather than application-level checks. On a duplicate-key error the handler falls back to update. No distributed locks are used.
- **Files** deduplicate on SHA-256 checksum. Two documents may legitimately reference one media item.
- **Scheduled workflows** take an atomic lock (`src/lib/workflowLocks.ts`), so an overlapping run skips rather than doubling up.

5.8 Scheduled and triggered tasks

Task	Plugin	Retries	Trigger
`pullSource`	per connector	3	Schedule, pull connectors only
`fetchDocument`	core	2	On document create with a pending fetch
`extractText`	core	2	On `fetched`
`extractCandidates`	ai	2	On `extracted` , when AI is enabled
`runImport`	core	0	On import commit
`submitForVerification`	verification adapter	3	On claim create, if the adapter pushes
`pullVerdicts`	verification adapter	3	Schedule, for reconciliation
`recomputeClaimStatus`	core	3	On claim-update change
`cascadeStatus`	core	3	On `recordStatus` change
`sweepStuck`	core	1	Every 5 min

Task	Plugin	Retries	Trigger
`cleanupJobs`	core	1	Hourly

Every task is wrapped in the Sentry tracing helper carried over from `src/tasks/utils.ts`, giving each a span, a duration, structured start, complete and error logs, and the ids needed to trace one document through the pipeline.

---

6. Error handling standards

6.1 Error hierarchy

```
class PromiseTrackerError extends Error {
  constructor(
    message: string,
    readonly code: string,
    readonly context: Record<string, unknown> = {},
    readonly retryable = false,
  ) { super(message) }
}

class ConfigurationError extends PromiseTrackerError {}   // fatal at boot, never retried
class PortResolutionError extends ConfigurationError {}   // zero or many adapters for a port
class ExternalServiceError extends PromiseTrackerError {} // carries status and provider id
class ExtractionError extends PromiseTrackerError {}      // text or AI extraction failed
class ValidationError extends PromiseTrackerError {}       // payload or row failed its schema
class OwnershipError extends PromiseTrackerError {}        // a write to a plugin-owned field
```

Every error carries a stable `code` for alerting, the ids needed to find the record, and an explicit `retryable` flag. Retry decisions read the flag; they never pattern-match on the message.

6.2 Logging standards

Logs are structured JSON. Every entry carries level, message, timestamp, source (for example `task.extractText` or `plugin.superdesk.push`), and the ids needed to trace one document end to end: `documentId`, `claimId`, `countryId`, `taskId`, `workflowRunId`, `plugin`. Errors carry code, stack and context. Any operation over 100ms carries a duration.

Rules:

1. Log to the console and to Sentry.
2. Always include the relevant ids.
3. Log success paths as well as failures; success logs establish the baseline anomaly detection needs.
4. Never log an API key, token, password, submitter email address or raw IP address.

6.3 Error recovery strategies

Failure	Where it surfaces	Recovery
A document fails extraction	`processing.status = 'failed'`, message on the record, email to the uploader	The uploader replaces the file or the link
AI returns no verifiable candidate	`processing.status = 'needs-review'`	A human reads the document and enters claims by hand
A verification provider is unreachable	Circuit breaker, health `degraded`	Claims queue locally; the reconciliation poll catches up
An inbound push arrives for an unknown record	`200` plus a recorded reconciliation item	The next poll resolves it
An import row cannot resolve a reference	Row error in the `ImportJob` preview	The editor fixes the sheet or creates the missing record
A plugin-owned field is written directly	`OwnershipError`, rejected	The write goes to the owning system
A port has zero or several adapters	`PortResolutionError` at boot, process exits	Deployment configuration is corrected

An error an editor can fix appears in the admin on the record itself. An error only an engineer can fix goes to Sentry and does not interrupt editorial work.

---

7. Frontend architecture

7.1 Rendering and data access

The admin and the public site share one Next.js deployment. Public pages are server-rendered and cached by tag (§11.3); none of them depends on an external service. Query `depth` is specified per query rather than globally; v2 uses `depth: 6` on page loads, walking relationships six levels deep on every render.

7.2 Page composition

Claim page, `/{cc}/promises/{slug}`**.**

1. Header: status chip in the colour from the status record, claim type, category, date made, target date if any.
2. The statement: the verbatim quote, visually distinct, attributed to the promisor, linking to the source document and page.
3. Context: the editorial summary.
4. Timeline: every published claim update in reverse-chronological order, with date, status at that date, narrative and evidence links.
5. Verification: verdict, who verified it and when, the rationale, and any published fact-check report.
6. Map: where the claim applies, when `jurisdiction` or the promisor's office carries geodata.
7. Actions: share, and submit an update.
8. Related: other claims by the same actor in the same category.

Actor profile, `/{cc}/politicians/{slug}` **and siblings.** Scorecard first: total claims and the count and share in each status group, rendered as a chart grouped by `ClaimStatus.group`. Then office history for a person, current office, party, and a filterable claim list. A map shows `person.originPlace` and `offices[].jurisdiction`.

Country landing, `/{cc}`**.** Country map with regions, headline numbers, featured actors, recently updated claims, and entries into each actor listing.

7.3 Listings and filtering

- Server-side pagination, 24 per page, cursor or offset.
- Server-side filtering on status group, category, actor, claim type and date range.
- Facet counts from aggregation rather than from the loaded page.
- Filter state in the URL (`?status=kept&category=health&page=2`), so a filtered view is shareable, bookmarkable and indexable.
- Sorting by most recently updated, deadline or status.

7.4 Search

- MongoDB text indexes over `claims` (`title`, `statement`, `summary`) and `actors` (`name`, `shortName`, `description`), weighted toward titles and names.
- Scoped by country by default, with an all-countries option.
- Results grouped by type: claims, politicians, parties, institutions, places.
- Server-rendered at `/{cc}/search?q=`, so results are linkable and crawlable.
- Isolated behind one module, so a later move to Atlas Search or Elasticsearch touches one file.

7.5 SEO and structured data

Feature	Implementation
Sitemaps	A sitemap index at `/sitemap.xml`, one child per country, split at 50,000 URLs
`robots.txt`	Generated, pointing at the sitemap index
Canonical URLs	On every page, absolute, from a configured `metadataBase`
OG images	Generated per claim and per actor through `next/og`, cached
ClaimReview JSON-LD	On every verified claim page

Feature	Implementation
Organization, Person and Place JSON-LD	On actor profiles
Breadcrumbs	`BreadcrumbList` JSON-LD matching the typed URL structure
RSS and Atom	Per country, for recently published and recently updated claims

ClaimReview is schema.org's standard for fact-checks and is what fact-check search surfaces consume. It is also the interoperability format the fact-checking community uses, which matters because Superdesk emits NINJS rather than ClaimReview.

```
{
  "@context": "https://schema.org",
  "@type": "ClaimReview",
  "url": "https://promisetracker.org/ke/promises/200-health-centres",
  "claimReviewed": "We will build 200 new health centres by 2027",
  "itemReviewed": {
    "@type": "Claim",
    "author": { "@type": "Person", "name": "William Ruto" },
    "datePublished": "2022-06-30"
  },
  "author": { "@type": "Organization", "name": "PromiseTracker" },
  "reviewRating": {
    "@type": "Rating",
    "ratingValue": 2, "bestRating": 5, "worstRating": 1,
    "alternateName": "In progress"
  }
}
```

The numeric rating derives from `ClaimStatus.group`; `alternateName` is the localized label.

7.6 Maps

Maps render from `GeoRef` on the country, the place, or the promisor's office jurisdiction. Degradation is explicit: boundary polygon, then a marker at `centroid`, then no map. A record with no geodata shows no map and nothing else changes.

Maps are client-side, lazily loaded, below the fold and excluded from LCP; the page is fully readable without one. Base tiles come from a configured provider with attribution. **CONFIRM** the provider and its licence for commercial and embedded use before implementation.

Maps appear on the country landing (regions, optionally shaded by performance), a place profile (its own boundary and children), an actor profile (origin and jurisdiction), and a claim page (the jurisdiction it applies to).

7.7 Internationalisation

Content locales are `en` and `fr` per country. The site carries a visible language switcher, locale as a query parameter or cookie-backed preference, and `hreflang` alternates on every page. All UI strings are localized; v2 hardcodes English filter and sort labels in `src/components/Promises/index.tsx`.

7.8 Accessibility

Requirement	Standard
Contrast	WCAG AA, including every status chip colour
Keyboard	Every filter, timeline control and map control reachable
Screen readers	The timeline is a semantic ordered list, not a decorative SVG
Status colours	Editor-configurable, so the admin shows a live contrast check against the chosen text colour and warns below AA

---

8. Testing requirements

Layer	Share	Covers
Unit	around 80%	Pure functions: `_status` computation, slug rules, quote verification, import mapping, retry and backoff
Integration	around 15%	Tasks with mocked externals, route handlers, access control, the cascade, plugin resolution
End to end	around 5%	Upload, extract, review, verify, publish, public page; and submission, moderation, timeline

Coverage targets: 70% overall, 85% on the pipeline, governance and plugin resolution, 90% on utilities.

Two rules specific to this architecture:

1. **Every plugin is tested enabled and disabled,** and a disabled plugin must return the incoming config byte-identical.
2. **Core is tested entirely on its own,** with no credentials for any external service. If that suite requires an API key, the standalone-core constraint (§1.4) has been broken and the build fails.

The v2 security suite carries over as the regression floor: `tests/int/{ssrf,fileSignature,requestGuards,securityHeaders,privilegedAccess,publicApiAccess,publicRoutes}.int.spec.ts`.

Test file naming follows the existing convention: `tests/int/**/*.int.spec.ts` for Vitest, `tests/e2e/**/*.e2e.spec.ts` for Playwright.

---

9. Security requirements

9.1 Authentication and authorization

Surface	Auth
Public site and public API	None
Embed routes	None
Admin	Payload session
Plugin inbound routes	A per-route shared secret, compared in constant time
Internal job trigger	Payload session

Collection access functions enforce roles. Field-level access guards `recordStatus`, `verification.*` and every plugin-owned field.

9.2 Input validation

A schema validates every piece of external data before use: webhook payloads, API responses, import rows, form submissions. Nothing reaches a database write unvalidated.

Inbound push handlers follow a fixed order:

1. Verify the signature before parsing the body.
2. Check the idempotency key against what has already been processed.
3. Validate the payload shape.
4. Only then touch the database.

9.3 Secrets management

Secret	Location	Rotation
`PAYLOAD_SECRET`, `DATABASE_URI`	Environment	Annually
S3 credentials	Environment	Annually
Plugin API keys and tokens	Settings global, encrypted at rest	On offboarding
Inbound webhook secrets	Environment	Every 6 months

`src/fields/encryptedSecret.ts` carries over unchanged: encrypt on write with a version prefix, return a mask to non-local callers, decrypt only for the Local API, and preserve the stored value when the mask is resubmitted, which prevents a careless admin save from wiping a key.

The Superdesk service account requires a real user account with the `archive` privilege, because the auth server grants no write scope. Scope it to the fact-check desk and name it recognisably, such as `promisetracker-bot`, so its edits are attributable in Superdesk's own history.

9.4 Outbound requests (SSRF)

By default the backend makes no outbound request to a user-supplied URL. `Document.sourceUrl`, `Evidence.url` and `Submission.evidenceUrl` are attribution only: stored, displayed and linked, never fetched.

The single exception is the `url` source connector, which runs through the guard in `src/utils/ssrf.ts` and `src/utils/files.ts`:

- HTTPS only; every other scheme is rejected.
- The hostname is resolved before the request and rejected if any resolved address is private (`10/8`, `172.16/12`, `192.168/16`), loopback, link-local including the `169.254.169.254` cloud-metadata endpoint, CGNAT, multicast or reserved, along with the IPv6 equivalents `::1`, `fe80::/10`, `fc00::/7`, IPv4-mapped, NAT64 and 6to4. IP-literal URLs are checked directly.
- Redirects are followed manually, every hop re-validated before it is dialled, under a hop limit.
- Timeouts per hop and in total; a byte cap enforced from `Content-Length` and again while streaming; a global concurrency limit.

- MIME validation plus magic-byte signature verification. SVG uploads are rejected outright.

Residual risk: DNS is validated immediately before each request, so a malicious authoritative server with a 0-TTL record could attempt a rebinding race in the window between validation and connection. The hosting environment's egress policy is the mitigation.

9.5 Uploads

- The extension and the declared MIME type must both be allowed and must agree, and magic bytes must match the claimed type.
- Size caps per kind.
- A rate limit and concurrency gate per client.
- A rolling daily byte budget bounding storage and bandwidth cost.
- Files are stored by content hash, so identical uploads deduplicate.

9.6 Data sanitization

Extracted text and any HTML from an external source is sanitised before storage, with an allowlist of tags and no attributes. Rich text authored in the admin is sanitised on render. Raw IP addresses are never stored; submissions keep a salted hash, sufficient to detect abuse patterns and insufficient to identify a contributor.

9.7 Roles and permissions

Roles are additive: a user with `contributor` and `reviewer` can do both.

Role	Purpose
`super_admin`	Full access. Manages users, settings, plugin credentials and taxonomies. The only role that can override a plugin-owned field, and that override is audit-logged
`editor`	Final content authority. Full `recordStatus` control on every collection. Owns the claim timeline
`contributor`	Adds actors, documents and draft claims. Cannot publish; may set `recordStatus` to `hold` only
`reviewer`	Triages extraction candidates and sets `verification.verdict`. Disabled when a verification plugin owns that field
`moderator`	Works the submissions queue and may promote an accepted submission to a claim update
`viewer`	Read-only

Action	super-admin	editor	contributor	reviewer	moderator	viewer
Create or update countries	✓	✓				
Create or update actors	✓	✓	✓			
Create or update documents	✓	✓	✓			
Create claims manually	✓	✓	✓			
Accept or reject extraction candidates	✓	✓		✓		
Set `verification.verdict`	✓	✓		✓		
Create or update claim updates	✓	✓			✓ from a submission	
Set `recordStatus`	any value	any value	`hold` only	on claims, any		
Moderate submissions	✓	✓			✓	
Hard-delete	✓	✓				
Override a plugin-owned field	✓, logged					
Manage users and settings	✓					
Read all records	✓	✓	✓	✓	✓	✓

Access is not scoped by country in v3. Any authenticated user with a write role can edit any country's records, and `User.countries` is an unenforced relationship. Enabling it later is a `where` clause in the collection access functions plus a field-level guard, with no schema change, since country is already a required, indexed field on every content collection.

9.8 Content security policy

The main site keeps a strict policy, including `frame-ancestors 'self'`.

`/embed/*` is the single exception: `frame-ancestors *` on those routes alone, serving pages that carry no cookies, no authentication and no state. The split keeps the admin and the main site un-embeddable while permitting syndication.

9.9 Public endpoint protection

Endpoint	Protection
Submission form	Captcha, per-IP rate limit, honeypot field, salted IP hash
Public API	Per-IP rate limit, CDN cache, pagination caps
Embeds	No cookies, no state, CDN-cached
Inbound plugin routes	Shared secret, per-IP rate limit, idempotency cache
Search	Query length cap, rate limit

9.10 Data retention

Data	Retention
Claims, updates, actors, documents	Indefinite; archived, never auto-deleted
Extraction `rawResponse`	90 days, then pruned. Candidates and decisions are kept
Rejected and spam submissions	180 days
Submitter contact details	Deleted on request; never published without `consentToAttribute`
Completed jobs	Deleted on completion
Failed jobs	7 days
Logs	Per the Sentry plan

---

10. Deployment and operations

10.1 Configuration validation

At boot, before serving:

1. Core environment variables present.
2. `PT_PLUGINS` resolves; dependencies satisfied; no cycles.
3. Every exclusive port has exactly one implementation.
4. `requiredEnv` present for each enabled plugin.
5. Database and object storage reachable.
6. Each enabled plugin's `validateSettings()` runs. The Superdesk adapter checks its profile, vocabulary, desk and stage ids against the live instance.

Any failure prints every problem at once and exits non-zero. A half-configured deployment that starts and then fails per request is worse than one that does not start.

10.2 Deployment topology

- Node 24, pnpm, a single application container. Tika runs in its own container when enabled.
- MongoDB and S3 are external managed services.
- Zero-downtime rolling deploys, with the health check gating readiness.
- Schema migrations are versioned, reversible where possible, and run before the new version serves traffic.
- Rollback triggers: error rate over 10% for 5 minutes, broken critical functionality, data corruption.

10.3 Observability

Structured logging and metric definitions are in §6.2. Metrics reported to Sentry: documents processed by stage, extraction candidates proposed and accepted, verification round-trip time, cascade job duration, push deliveries received and rejected, public API request rate, cache hit ratio.

10.4 Health checks

`/api/health` reports the resolved configuration and every enabled dependency.

```
{
  "status": "healthy | degraded | unhealthy",
  "timestamp": "...",
  "plugins": { "enabled": ["core","tika","ai","superdesk"],
               "ports": { "verification": "superdesk", "extractors": ["plaintext","tika"] } },
  "services": {
    "database":  { "healthy": true, "latencyMs": 4 },
    "storage":   { "healthy": true, "latencyMs": 22 },
    "tika":      { "healthy": true, "latencyMs": 31 },
    "llm":       { "healthy": true, "latencyMs": 180 },
    "superdesk": { "healthy": true, "latencyMs": 95 }
  }
}
```

`/api/health/ready` covers only the database and object storage. Extractors, AI and the verification provider report into `/api/health` and degrade the pipeline, not the site. In v2, Tika is an unconditional readiness dependency, so a Tika outage marks the whole application unready even though no public page touches it.

10.5 Alerting

Condition	Route
Pipeline success rate below 90% for 15 min	Slack
A dependency unhealthy for over 5 min	PagerDuty
Error rate above 5% for 10 min	Slack

Condition	Route
Push signature failures spike	Slack
Content API empty for 3 consecutive polls	Slack
AI credits exhausted	Slack, plus health `degraded`
Database p95 over 500ms	Slack warning

Editors are never paged for a failure only an engineer can act on.

10.6 Backup and recovery

The database takes a daily full backup with 30-day retention on versioned object storage. Media uses bucket versioning plus cross-region replication. RTO is 4 hours and RPO is 24 hours. Restores are tested quarterly.

10.7 Initial data seeding

A new deployment requires configuration records before any content can exist. This is the dependency order.

Order	Collection	Why it comes first	Minimum
1	`claim_statuses`	Every claim update references one, and `group` drives every scorecard	One full set, every record with a `group`
2	`categories`	Claims filter and chart by category	8 to 15, localized
3	`countries`	The first path segment; nothing public resolves without one	One per country
4	`users`	Roles gate every other step	One super-admin, one editor per country
5	`site_settings`	Navigation, footer, branding	Enough for a page to render
6	`actors`	Claims require a promisor	The actors being tracked

`ClaimStatus.group` has no upstream source and no sensible default. Scorecards are wrong rather than empty if it is missing or inconsistent, so it is assigned once, deliberately, before the first claim is published.

Documents, extractions and claims follow the ordinary pipeline and need no special import path.

---

11. Performance requirements

11.1 Response time targets

Operation	Target
Inbound push handler	< 5s p95
Public API	< 300ms p95
Health check	< 100ms
Document fetch	< 5 min (timeout)
Text extraction	< 2 min per document
AI extraction	< 3 min per document
Full pipeline per document	< 15 min p95
Cached country landing or claim page, at the origin	< 100ms
Frontend LCP	< 2.5s on 4G
Frontend TTI	< 3s
Frontend CLS	< 0.1

11.2 Scalability targets

	Phase 1	Phase 2	Phase 3+
Countries	5	20	50+
Actors	200	2,000	10,000
Claims	2,000	25,000	150,000
Claim updates	5,000	75,000	500,000
Documents	500	10,000	50,000
Monthly visitors	20,000	200,000	1,000,000

The mechanisms that satisfy each axis:

Axis	Mechanism
More countries	A country is one record with a code. Creating it yields a landing page, listings and a sitemap with no deploy and no configuration. There is no tenancy to provision
Traffic spikes	Tag-based caching with precise `revalidateTag` on publish, `generateStaticParams` for published records, CDN-cacheable embeds and API, immutable media headers, and a read path with no external dependency
Volume	Server-side pagination and filtering with aggregation facets, compound indexes on every listing query, text indexes for search, `depth` specified per query, and a cascade job that batches rather than writing per record
Editorial throughput	One indexed `processing.status` as the work queue, a candidate review queue with per-row decisions, Payload document locking, roles separating contribution from publication, and a moderation queue outside the editorial path

11.3 Caching strategy

Tag	Invalidated when
`country:{code}`	The country record changes
`actor:{id}`	That actor, or a claim naming it, changes
`claim:{id}`	The claim or any of its updates changes
`claims:{country}`	Any claim in that country is published or unpublished
`statuses` , `categories`	The taxonomy changes
`settings`	Site settings change

Page data is wrapped in `unstable_cache` with the relevant tags and a long `revalidate` ceiling as a backstop. `generateStaticParams` prerenders country landings, actor profiles and claim pages for published records; the long tail renders on demand and is then cached. The cascade job calls `revalidateTag` on completion, so publishing propagates in seconds rather than waiting for a timer. Media is served with `Cache-Control: public, max-age=31536000, immutable`.

11.4 Frontend budgets

Budget	Limit
JavaScript on a claim page	< 150 KB gzipped, excluding the lazily loaded map
Images	Next.js image optimisation, explicit dimensions, no layout shift
Map	Client-side, lazily loaded, below the fold, excluded from LCP

---

12. Development workflow

12.1 Repository and branches

`main` is the production branch. Work happens on prefixed branches (`ft/` , `fix/` , `chore/`) and merges by pull request. Commit messages follow Conventional Commits, as the existing history does.

12.2 Commands

Task	Command
Install	`pnpm i`
Development server	`pnpm dev`
Clear the dev cache	`pnpm devsafe`
Build	`pnpm build`
Start	`pnpm start`
Lint	`pnpm lint`
Integration tests	`pnpm test:int`
End-to-end tests	`pnpm test:e2e`
All tests	`pnpm test`
Payload types and import map	`pnpm generate:all`
Tika, for local extraction	`docker compose up -d`

Continuous integration runs `pnpm lint` , `pnpm test:int` and `pnpm test:e2e` with `PT_PLUGINS=core` and no credentials in the environment.

12.3 Delivery phases

Phase	Contents	Complete when
1. Core	Kernel, plugin contract, registry, boot validation, every collection in §4, `recordStatus` and the cascade, roles, upload and URL connectors, plaintext and HTML extraction, CSV and XLSX import, native review queue, public site with typed routes, caching, pagination, filtering and search, sitemaps and ClaimReview, submission form and moderation queue	A non-developer creates a country, enters an actor and a claim by hand, adds three dated updates, publishes, and the public page renders the timeline, with no API key in the environment
2. Scale and assist	`plugin-tika` , `plugin-ai` with one provider, candidate review UI, maps, embeds and the public read API, CSV export, language switcher, per-country editorial scoping	Country scoping is enforced, and a 200-page PDF produces reviewed claims
3. Newsroom	`plugin-superdesk` , instance configuration and validation, HTTP-push receiver, reconciliation poll, verdict round-trip, switch of `PT_VERIFICATION` from `native`	Verdicts set in Superdesk appear on published claims, and a killed item depublishes

Per-country editorial scoping is first in phase 2 rather than last: it is a `where` clause and a field guard with no schema change, and it must exist before an external newsroom receives an account.

---

13. API documentation

13.1 Public read API

```
GET /api/public/v1/countries
GET /api/public/v1/{cc}/actors?kind=person&page=1
GET /api/public/v1/{cc}/claims?status=kept&category=health&updatedSince=2026-01-01
GET /api/public/v1/{cc}/claims/{slug}
GET /api/public/v1/{cc}/claims/{slug}/updates
```

- Auth: none. Rate-limited per IP, cached at the CDN.
- Published records only, under the same `_status` filter as the site, so "public" has one definition.
- Stable, documented JSON, deliberately not the raw Payload REST surface. v2 exposes `/api/[...slug]` with `read: publishedOrAuthenticated` on promises, which makes the internal document shape the public contract.
- `Cache-Control` and `ETag` on every response.
- OpenAPI description at `/api/public/v1/openapi.json`.
- Toggled by `Settings.features.publicApiEnabled`.

CSV export of claims and updates is generated on demand from a live query, filtered by the same parameters.

13.2 Embed routes

```
GET /embed/claim/{id}           A single claim card with status and last update
GET /embed/scorecard/{actorId}  An actor's status breakdown
GET /embed/list?...             A filtered claim list
GET /api/oembed?url=...         oEmbed discovery for a claim URL
```

Served only from `/embed/*`, with `frame-ancestors *` on those routes alone. No cookies, no tracking, and no JavaScript beyond an auto-height `postMessage`. A plain `<iframe>` works without the sizing script. Each embed carries a visible link to the canonical page. Toggled by `Settings.features.embedsEnabled`.

13.3 Core endpoints

```
GET  /api/health          Auth: none.    Full dependency and plugin report (§10.4)
GET  /api/health/ready    Auth: none.    Database and storage only
POST /api/submissions     Auth: none.    Rate-limited, captcha, honeypot. Creates a Submission
GET  /sitemap.xml         Auth: none.    Sitemap index
GET  /{cc}/feed.xml       Auth: none.    Per-country RSS
```

13.4 Plugin endpoints

These routes exist in the filesystem at all times. When their plugin is disabled they return `503` before executing any application logic (§3.10).

```
POST /api/plugins/superdesk/items
     Auth: x-superdesk-signature, HMAC-SHA1 over the raw body, verified in constant time
     Body: a NINJS item
     200 even for an unknown guid, so Superdesk stops retrying; the item is recorded
          for reconciliation
     401 on an invalid signature, logged at warn, body never parsed

GET  /api/plugins/superdesk/assets
POST /api/plugins/superdesk/assets
     Auth: as above. Media exchange for pushed items

POST /api/plugins/airtable/webhook
     Auth: bearer shared secret
     Idempotency: delivery id checked against the last-seen id on the target record
```

14. Design decisions

Why is the core a plugin rather than the application? If core were the application and plugins were additions, every integration would reach into application internals and the boundary would erode. That is how v2 came to write Airtable status strings from six pipeline tasks. Making core a plugin forces it to publish the same extension API every integration uses, so an extension point that is inadequate for an integration is inadequate for core too. It also keeps core replaceable in principle: a different domain model swaps `plugin-core` and keeps the kernel, the port interfaces and the integrations.

Why must the system run with no external service? Because the alternative is what v2 became. A configuration that cannot run without two third-party services cannot be tested in CI, cannot be handed to a new contributor, and cannot survive either vendor changing terms. Making the no-credential configuration a complete product, and the configuration CI runs, keeps that property from eroding.

Why do ports fail at boot rather than returning 503? A disabled optional endpoint returning `503` is correct, and plugin routes still behave that way. A required capability with interchangeable implementations is different: a deployment with no verification provider, or two, is misconfigured, and the failure should surface at deploy time with a message naming the candidates, not per request in production.

Why are plugin-owned fields read-only in the admin? To guarantee a single writer. If both the admin and an external system can write the same field, conflicts are inevitable and attributing a change becomes guesswork. Read-only keeps the data visible for debugging while making the direction of flow unambiguous.

Why are injected plugin fields always optional? A plugin may be removed after months of use. Required fields would invalidate every existing record on removal. Optional fields degrade gracefully: the data stays in the database, the system keeps working, and the fields can be cleaned up separately.

Why environment variables for plugin selection rather than database flags? Plugins change the Payload config at build time through field injection and route declaration. A database flag is read at runtime, too late for either. Environment variables also make plugin state explicit in the deployment configuration.

Why do credentials live in the database rather than in environment variables? They change per deployment without a code change, and an encrypted field already exists that encrypts on write, masks on read, and restricts access to super-admins. Environment variables select which plugins run; the database holds what they need to run.

Why both an HTTP push and a reconciliation poll? Pushes are fast but not guaranteed: a deploy, a network fault or a disabled subscriber loses them silently, and the Superdesk Content API returns `200` with an empty list when a subscriber matches no product. The poll bounds the worst case to the poll interval.

Why is country a path segment rather than a tenant? A tenant boundary costs an origin, a certificate, a cookie domain and a cache per country, makes cross-country pages impossible, and in v2 provided no isolation at all (`userHasAccessToAllTenants: () => true`). A path segment gives one origin, one cache, and cross-country queries, while country remains a required indexed field, which is all that access scoping needs when it is added.

Why typed URL segments for actors? A flat `/{cc}/{slug}` forces slug uniqueness across every actor kind and requires a lookup across all of them before the server knows what it is rendering. Typed segments scope uniqueness to `(country, kind)` and let `/ke/politicians/nairobi` and `/ke/places/nairobi` coexist.

Why one actors collection with a kind instead of separate collections? A person, a party, a ministry and a county all make promises. Separate collections would mean three further route trees, three further admin surfaces and a union query for every question about promises in a country.

Why is a claim's current status derived rather than stored? A stored status is a fact without provenance. Deriving it from the most recent published update means the status, its date, its narrative and its evidence always agree, and the timeline cannot be fiction. v2 stored the status and rendered its timeline from an empty array.

Why two publication fields rather than one? A single checkbox cannot express "this claim is fit to publish but the document it came from is under review". Separating intent from outcome lets a parent be withdrawn without losing each child's individual intent, and lets release restore exactly the children that were only hidden by the parent.

Why does a claim's source document count as an ancestor? The document is the evidence. If a manifesto turns out to be inauthentic, every claim extracted from it must leave the site with it, and that should require one action rather than a sweep.

Why verify quotes against the source text? Because an extraction model will otherwise produce plausible claims that nobody made. Discarding any candidate whose quoted span does not literally appear in the document is the difference between an assistant and a fabrication engine, and it costs one string search.

Why three ingestion ports rather than one? A partner's CSV of claims and a 200-page manifesto are different problems. One is a row-to-record mapping needing no language model; the other is text extraction followed by AI extraction and human review. v2 had one path and assumed PDFs.

Why store boundary polygons rather than fetch them? Boundary datasets carry licence terms, change rarely, and are large. A live tile or geometry dependency would reintroduce exactly the third-party coupling this architecture removes, and would break the map when someone else's terms change.

Why Payload CMS? A built-in admin, a plugin API strong enough for the field injection and ownership rules in §3, native TypeScript types, a job queue for the pipeline, and first-class Next.js integration.

Why MongoDB? A flexible schema for an evolving model, good document-query performance, and Payload's best-supported adapter. It is also already in production.

Why self-hosted Apache Tika? Open source, no per-call cost, over 1,000 supported formats, and no data leaving the deployment for text extraction.

Why keep AI extraction at all? A 200-page manifesto contains a few hundred candidate commitments. Reading each one is the bottleneck in the work, and a model that proposes candidates with verified quotes removes most of that reading without removing the human decision.

---

15. Appendix A: items to confirm

Each item below is marked **CONFIRM** in the body. Items under Superdesk require access to the newsroom's instance and are prerequisites for phase 3; the remainder are prerequisites for the features named.

A.1 Superdesk instance

1. The Content API prefix, and whether the Content API is enabled. Derived from `CONTENTAPI_URL`; it may be `/v1`, `/capi` or another value. §3.7.2
2. Whether the Production API is deployed. If absent, reading unpublished items requires the internal API and a session token. §3.7.2
3. The content profile id for claim or fact-check items, and whether one exists. §3.7.2
4. The vocabulary ids and `qcode` values for verdict, claim type and category, and whether verdict is modelled as a `subject` scheme or an `extra` field. §3.7.2
5. The desk and stage ids for the fact-checking desk. §3.7.2
6. A service account with the `archive` privilege, and the newsroom's agreement to it. §9.3
7. Whether `http_push` is available, and whether a subscriber and product can be configured. If not, polling is the only inbound path and latency rises to the poll interval. §3.7.2
8. The deployed Superdesk version. The published changelog stops at 2.4.5 (2022) and the codebase is mid-rewrite; `content_api/items` still carries a legacy auth shim. Pin the version and re-verify the query surface. §3.7.2
9. Whether `extra.*` is mapped in Elasticsearch on this instance. §3.7.2
10. Whether content-list webhooks exist in the deployed version. They are present on `develop` and absent from the published documentation. §3.7.2

A.2 Taxonomies

11. The verdict vocabulary. v2 inherits Meedan's `unverified / verified / false / misleading`, which answers whether a statement is true. A promise tracker more often asks whether a promise is accurately attributed and whether it was kept. Decide before the Superdesk vocabularies are created. §4.1.3
12. The claim status set and the `group` assigned to each. Prerequisite for publishing any claim. §10.7
13. The category list. Prerequisite for publishing any claim. §10.7
14. Whether the four claim types in §4.1.3 are right, and whether each needs distinct fields on the public page.

A.3 Geospatial

15. The map tile provider and its licence for commercial and embedded use. Prerequisite for maps. §7.6
16. The boundary data source, whether geoBoundaries, GADM or national sources, and its attribution requirements. Prerequisite for maps. §7.6
17. The topology-preserving GeoJSON simplification step for large boundaries. §4.5

A.4 Capacity

18. The volume at which MongoDB text indexes stop being adequate for search, and whether Atlas Search is available on the hosting plan. §7.4